

UAB

Universitat Autònoma de Barcelona

Diseño e implementación de una API de control de acceso para la gestión de usuarios y organizaciones

Segundo informe de seguimiento

Igor Ulyanov Melnic
Tutora: Victoria Montenegro Ruíz

Grado en Ingeniería Informática
Escuela de Ingeniería
Curso 2025-2026

0. Control del documento

0.1 Información del documento

Identificador	segundoSeguimiento
Autor	Igor Ulyanov Melnic
Fecha de creación	15/5/2026
Ultima modificacion	21/5/2026
Nombre del fichero	Informe de Seguimiento 2.pdf

0.2 Historial del documento

Versión	Fecha	Cambios
1.0	16/5/2026	Creación del documento, esbozo inicial
1.1	17/5/2026	Refinamiento del esbozo inicial
1.2	19/5/2026	Finalización de primera version
1.3	21/5/2026	Añadido diagramas secuencia

Índice

0. Control del documento.....	2
0.1 Información del documento.....	2
0.2 Historial del documento.....	2
Índice.....	3
1. Indicación del nivel de seguimiento de la planificación prevista y ajustes efectuados...	4
2. Exposición de los resultados.....	5
3.1 Inicialización del proyecto.....	5
3.2 Implementación de la estructura de la API.....	6
3.3 Implementación de operaciones CRUD con endpoints.....	10
3.4 Requests con DTOs.....	11
3.6 Validación y gestión de errores.....	12
4. Diagramas de secuencia.....	15
4.1 Peticion HTTP.....	15
4.2 Autenticación de usuario.....	16
4.3. Autorización de acciones.....	16
5. Conclusiones provisionales.....	17
6. Fuentes de información consultadas.....	17
7. Uso de la IA generativa en el desarrollo del informe y del trabajo.....	17

1. Indicación del nivel de seguimiento de la planificación prevista y ajustes efectuados

El desarrollo del proyecto ha continuado, en términos generales, conforme a la planificación establecida en el primer informe de seguimiento. Tal y como se definió previamente, el proyecto se encontraba en fase de implementación, y durante este periodo se ha avanzado de forma significativa en la construcción de la API backend.

El grado de cumplimiento de la planificación ha sido alto. Se han implementado las principales funcionalidades previstas para esta fase, especialmente aquellas relacionadas con la gestión de entidades fundamentales del sistema (usuarios, organizaciones y roles), así como su integración con la base de datos previamente diseñada .

En relación con el diagrama de Gantt inicialmente planteado (figura 1), el proyecto sigue el curso previsto, habiéndose completado todas las tareas asociadas a la implementación de las operaciones CRUD (Create, Read, Update y Delete) sobre los elementos clave de la base de datos, expuestos como endpoints de la API.

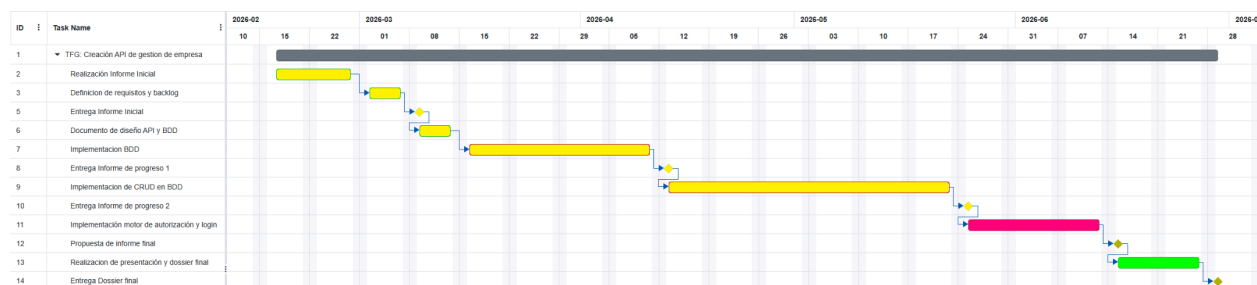


Fig 1. Diagrama de Gantt con progreso reflejado

Asimismo, se mantiene el uso de la herramienta Trello para la implementación de un tablero Kanban, que permite llevar un seguimiento detallado de las tareas completadas, las pendientes y los problemas identificados durante el desarrollo. En la figura 2 se muestra el estado actual de dicho tablero. En él se puede observar que la implementación de la base de datos en la API desarrollada con Spring se encuentra finalizada, incluyendo algunas mejoras no previstas inicialmente, como la personalización de los mensajes de error y, de cara a futuras iteraciones, una exposición más clara de los datos en las solicitudes GET. Asimismo, se reflejan las tareas aún pendientes, entre las que destacan la implementación de la lógica de negocio y el sistema de autenticación de usuarios.

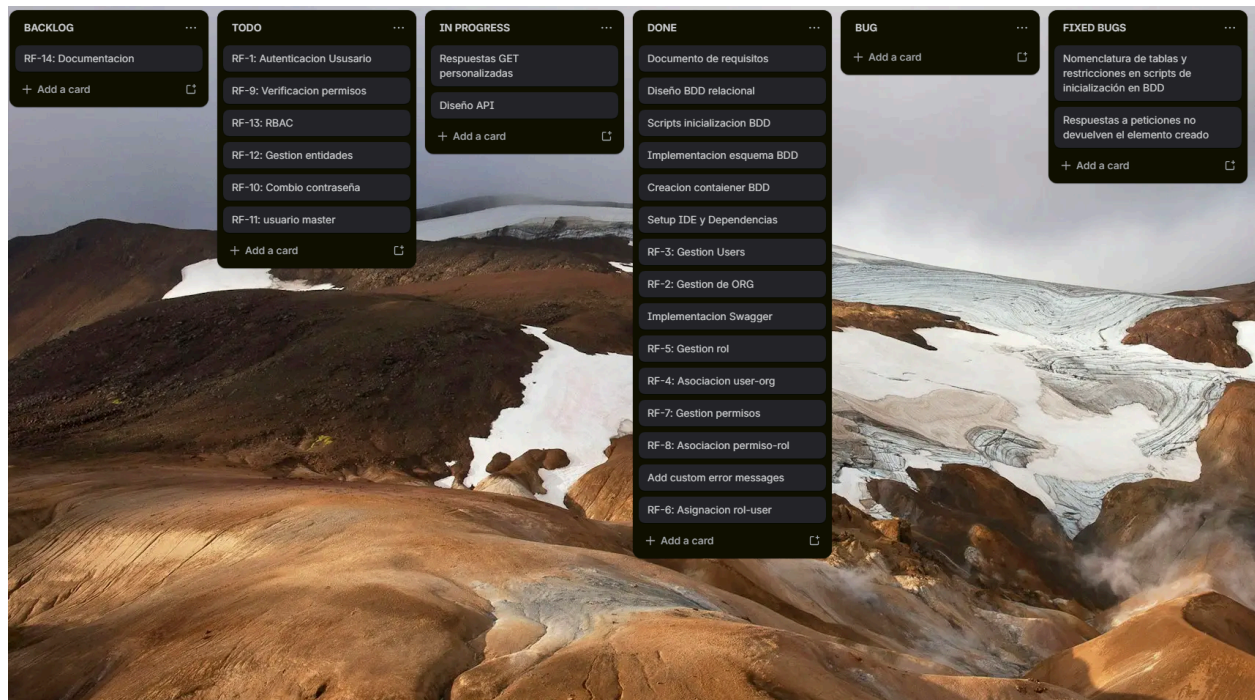


Fig 2. Tablero Kanban en su estado actual

2. Exposición de los resultados

3.1 Inicialización del proyecto

En esta fase se ha llevado a cabo la creación del proyecto utilizando el entorno de desarrollo IntelliJ IDEA con Maven, estableciendo una estructura inicial de carpetas acorde a las buenas prácticas en aplicaciones basadas en Spring^[1]. Asimismo, se han configurado los archivos necesarios para garantizar un correcto punto de partida del desarrollo.

Entre los elementos clave configurados, destaca el archivo *pom.xml*, en el que se han incluido las dependencias necesarias para el proyecto, así como el archivo *application.yaml*, donde se ha definido la configuración de la conexión con la base de datos desplegada en un contenedor. Además, se ha implementado la clase principal de la aplicación, anotada con *@SpringBootApplication*, que actúa como punto de entrada del sistema.

Con el objetivo de verificar el correcto funcionamiento inicial, se ha desarrollado un controlador de prueba que expone un endpoint GET, el cual devuelve una cadena de texto sencilla como respuesta. De forma complementaria, se ha ajustado la configuración de seguridad, desactivando temporalmente el sistema de autenticación para facilitar las pruebas en las primeras etapas del desarrollo.

Una de las dependencias más relevantes incorporadas en el proyecto es Swagger, utilizada para la generación automática de la documentación bajo el estándar OpenAPI^[3]. Esta

herramienta permite añadir anotaciones descriptivas sobre el funcionamiento y uso de cada endpoint, facilitando así su comprensión. Además, proporciona una interfaz visual que permite interactuar de forma sencilla con la API en tiempo real, lo que resulta especialmente útil durante las fases de desarrollo y pruebas.

3.2 Implementación de la estructura de la API

Siguiendo la arquitectura propuesta por Spring Framework, se han desarrollado las distintas capas de la aplicación, lo que permite establecer una separación clara y coherente de las responsabilidades dentro de la API. Esta organización facilita la mantenibilidad, escalabilidad y comprensión del sistema. El flujo de funcionamiento sigue la secuencia:

Controller → Service → Repository → Database.

A continuación, se describen las funciones de cada una de estas capas:

- **Controller:** Se encarga de gestionar las peticiones HTTP recibidas por la API. Actúa como punto de entrada del sistema, interpretando las solicitudes del cliente y delegando la lógica correspondiente a la capa de servicio.
- **Service:** Contiene la lógica de negocio de la aplicación. Procesa los datos, aplica las reglas necesarias y coordina las operaciones entre el controlador y el repositorio.
- **Repository:** Se ocupa del acceso a los datos. Proporciona una abstracción para interactuar con la base de datos, permitiendo realizar operaciones CRUD sin necesidad de escribir consultas SQL complejas de forma manual.
- **Base de datos:** Es el sistema de almacenamiento persistente donde se guardan los datos de la aplicación. Constituye la capa final del flujo y es accedida a través de los repositorios.

Esta estructura se basa en las buenas prácticas recomendadas en el ecosistema de Spring Framework^[2], donde se promueve la separación de responsabilidades en capas como Controller, Service y Repository dentro de aplicaciones web y APIs REST.

Para replicar la estructura de la base de datos en Spring y permitir, por tanto, la interacción del framework con la misma, se han definido una serie de clases y anotaciones que establecen el mapeo entre el código de la aplicación y el esquema de la base de datos. El elemento más básico de esta estructura corresponde a las clases de modelo.

Estas clases describen a Spring Framework la correspondencia entre los atributos de una clase Java y los campos de las tablas existentes en la base de datos PostgreSQL. De este modo, cada entidad del sistema queda representada de forma directa en el código. Por ejemplo en la figura 3 se muestra el fichero correspondiente al modelo de la tabla de usuarios que guardara la información pertinente además de métodos relevantes.

```

@Entity 29 usages
@Table(name = "users")
public class User {

    @Id 1 usage
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Username is required") 2 usages
    private String username;

    @Email(message = "Invalid email format") 2 usages
    @NotBlank(message = "Email is required")
    private String email;

    private String pass_hash; 2 usages

    public User() {} no usages

    public Long getId() { return id; }

    public String getUsername() { return username; }

    public void setUsername(String username) { this.username = username; }

    public String getPass_hash() { return pass_hash; }

    public void setPass_hash(String pass_hash) { this.pass_hash = pass_hash; }

    public String getEmail() { return email; }

    public void setEmail(String email) { this.email = email; }
}

```

Fig. 3: fichero User.java en el package authapi.model

En este tipo de clases destacan las siguientes anotaciones estándar de la especificación de persistencia en Java (Eclipse Foundation, 2022)^[4]:

- **@Entity**: indica que la clase representa una entidad persistente y se corresponde con una tabla de la base de datos.
- **@Table**: especifica el nombre concreto de la tabla a la que se enlaza la entidad.
- **@Id**: define el atributo que actúa como clave primaria de la tabla.
- **@GeneratedValue**: establece la generación automática del identificador de la entidad.

Las demás anotaciones están relacionadas con la validación de los datos de entrada siguiendo el estándar de validación de Beans^[5]. Además, estas clases incluyen los métodos *getter* y *setter* correspondientes a cada atributo, los cuales permiten el acceso y la manipulación de los datos. Su presencia es esencial para el correcto funcionamiento de la serialización y comunicación de las respuestas HTTP.

El siguiente nivel dentro de la arquitectura corresponde a los repositorios. Estas interfaces permiten a Spring generar consultas SQL de manera automática mediante una nomenclatura específica, reduciendo la necesidad de implementar consultas manuales. De este modo, se proporciona una capa de abstracción sobre el acceso a la base de datos, simplificando la lógica de persistencia y facilitando el desarrollo de la capa de servicios.

```
public interface UserRepository extends JpaRepository<User, Long> { 6 usages
    boolean existsByEmail(String email); 1 usage

    boolean existsByUsername(String username); 1 usage

    User findByEmail(String email); no usages

    User findByUsername(String email); 4 usages
}
```

Fig. 4: fichero *UserRepository.java* en el package *authapi.repository*

A continuación, se encuentran los servicios, que constituyen la capa donde se implementa la lógica de negocio de la aplicación. En estos ficheros se definen los métodos que serán invocados desde los controladores, encargándose del tratamiento de los datos de entrada, la aplicación de reglas de negocio y la preparación de las respuestas. Cada servicio integra los repositorios necesarios para acceder a la información requerida.

```
@Service 3 usages
public class UserService {

    private final UserRepository userRepository; 10 usages

    public UserService(UserRepository userRepository) { this.userRepository = userRepository; }

    public User createUser(User user) {...}

    public List<User> getAllUsers() { return userRepository.findAll(); }

    public void deleteUser(Long id) {...}

    public User updateUser(Long id, User updatedUser) {...}
}
```


Fig. 5: fichero UserService.java en el package authapi.service

Finalmente, la última capa fundamental de la API está formada por los controladores. Estos componentes se encargan de exponer los endpoints HTTP y de redirigir las peticiones a los servicios correspondientes. Las rutas pueden incluir parámetros de consulta o cuerpos de petición en formato JSON, que son procesados posteriormente por la lógica de negocio.

Asimismo, en estos controladores se incorporan anotaciones de Swagger con el objetivo de documentar y clarificar el funcionamiento de cada endpoint, facilitando así su comprensión e interacción.

```
@RestController no usages
@RequestMapping("/users")
@Tag(name = "Users", description = "User management endpoints for CRUD operations")
public class UserController {

    private final UserService userService; 5 usages

    public UserController(UserService userService) { this.userService = userService; }

    @Operation(summary = "Create a new user") no usages
    @PostMapping
    public User createUser(@Valid @RequestBody User user) {
        return userService.createUser(user);
    }

    @Operation(summary = "View list of all users") no usages
    @GetMapping
    public List<User> getAllUsers() { return userService.getAllUsers(); }

    @Operation(summary = "Delete a user by their id") no usages
    @DeleteMapping("/{id}")
    public void deleteUser(@PathVariable Long id) { userService.deleteUser(id); }

    @Operation(summary = "Edit a user's data by their id (partial editing accepted)") no usages
    @PutMapping("/{id}")
    public User updateUser(@PathVariable Long id,
                           @RequestBody User user) {

        return userService.updateUser(id, user);
    }
}
```

Fig. 6: fichero UserController.java en el package authapi.controller

3.3 Implementación de operaciones CRUD con endpoints

Para implementar un endpoint que realice una operación concreta, es necesario exponerlo a través de la API. Esto se lleva a cabo en la capa de controladores, donde se definen métodos públicos anotados con las correspondientes anotaciones de Spring Framework, como `@GetMapping` o `@PostMapping`, que permiten asociar cada función a un tipo específico de petición HTTP.

Adicionalmente, estos métodos pueden complementarse con anotaciones de Swagger, lo que facilita una representación más clara y detallada de la funcionalidad de cada endpoint dentro de la documentación generada automáticamente. En la figura 7 se puede observar un ejemplo de endpoint POST para poder agregar un usuario a la base de datos.

```
@Operation(summary = "Create a new user") no usages
@PostMapping
public User createUser(@Valid @RequestBody User user) {
    return userService.createUser(user);
}
```

Fig. 7: Funcion `createUser` en el fichero `UserController.java`

Estas funciones del controlador delegan la lógica de procesamiento en la capa de servicios, encargada de gestionar la información y generar una respuesta adecuada. Como se muestra en la figura 7, la anotación `@RequestBody` permite mapear automáticamente el cuerpo de la petición HTTP a un objeto Java, facilitando su tratamiento dentro de la aplicación.

En la capa de servicio, la implementación del método correspondiente incluye una serie de validaciones sobre los datos de entrada, seguidas de la invocación al repositorio para llevar a cabo la operación de persistencia. El método de guardado devuelve la entidad almacenada, lo que permite incluirla en la respuesta HTTP enviada al cliente. En la figura 8 se puede visualizar el código correspondiente a la creación de un usuario.

```

public User createUser(User user) { 1 usage
    if (userRepository.existsByEmail(user.getEmail())){
        throw new RuntimeException("Email already in use");
    };

    if (userRepository.existsByUsername(user.getUsername())){
        throw new RuntimeException("Username already in use");
    };

    return userRepository.save(user);
}

```

Fig. 8: Función `createUser` en el fichero `UserService.java`

Por último, la capa de repositorio proporciona de forma automática las operaciones básicas de acceso a datos, ya que las interfaces definidas extienden de *JpaRepository*. Esto permite disponer de funcionalidades CRUD sin necesidad de implementar manualmente las consultas SQL, simplificando considerablemente el desarrollo y mejorando la mantenibilidad del sistema.

3.4 Requests con DTOs

En determinadas peticiones a la API, sería necesario solicitar al usuario una cantidad elevada de información que, en muchos casos, no resulta intuitiva ni debería ser de su conocimiento. Por ejemplo, en el proceso de creación de un nuevo rol, el usuario únicamente debería proporcionar datos básicos como el nombre de la organización, el nombre del rol, la lista de permisos que desea asociar y, de forma opcional, una descripción.

Sin embargo, esta información no es suficiente para que el sistema pueda realizar la operación completa a nivel de base de datos, ya que es necesario gestionar claves primarias, relaciones entre entidades y otros datos internos. Estos elementos, aunque esenciales para el funcionamiento del sistema, resultarían complejos de manejar manualmente por parte del usuario, pero pueden ser resueltos de forma eficiente por la aplicación a partir de la información básica proporcionada.

Para abordar este problema, se emplean los denominados *Data Transfer Objects* (DTOs). Estos objetos, implementados en Java dentro del ecosistema de Spring Framework, proporcionan un nivel de abstracción que simplifica la interacción del usuario con la API. Los DTOs encapsulan únicamente la información mínima necesaria para llevar a cabo una operación, facilitando tanto la entrada de datos como su validación.

Además, los DTOs también pueden utilizarse en sentido inverso, es decir, para estructurar y simplificar la información devuelta al usuario en las respuestas HTTP. De este modo, se evita exponer directamente las entidades completas del sistema, mostrando únicamente los datos

relevantes. Esta funcionalidad, aunque no se encuentra implementada en la versión actual del proyecto, se plantea como una posible mejora futura orientada a optimizar la claridad y seguridad de las respuestas.

```
public class CreateRoleRequest { 6 usages

    @NotBlank(message = "Role name is required") 1 usage
    private String name;

    private String description; 1 usage

    @NotBlank(message = "A role must belong to an org, please input") 1 usage
    private String organizationName;

    @NotEmpty(message = "Permission list is required") 1 usage
    private List<String> permissions;

    public CreateRoleRequest() {} no usages

    public String getName() { return this.name; }

    public String getDescription() { return this.description; }

    public String getOrganizationName() { return this.organizationName; }

    public List<String> getPermissions() { return this.permissions; }

}
```

Fig. 9: DTO para la petición de creación de un rol

3.6 Validación y gestión de errores

Con el objetivo de proporcionar respuestas de error claras y comprensibles al usuario cuando una petición no es válida, resulta fundamental gestionar adecuadamente las excepciones generadas por el sistema. Para ello, se implementa un manejador global de excepciones, que permite capturar y procesar los errores de forma centralizada, devolviendo respuestas más personalizadas y adaptadas a cada situación. La implementación de este mecanismo se muestra en la figura 10.

```

@RestControllerAdvice no usages
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class) no usages
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public Map<String, String> handleValidationExceptions(
        MethodArgumentNotValidException ex) {

        Map<String, String> errors = new HashMap<>();

        ex.getBindingResult().getFieldErrors().forEach( FieldError error ->
            errors.put(error.getField(), error.getDefaultMessage())
        );

        return errors;
    }

    @ExceptionHandler(RuntimeException.class) no usages
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public Map<String, String> handleRuntimeException(RuntimeException ex) {
        Map<String, String> error = new HashMap<>();
        error.put("ERROR", ex.getMessage());
        return error;
    }
}

```

Fig. 10: GlobalExceptionHandler.java

Esta clase implementa un manejador global de excepciones en la aplicación mediante la anotación `@RestControllerAdvice` de Spring Framework^[6]. Su función principal es centralizar la gestión de errores que se producen durante la ejecución de la API, evitando la necesidad de manejar excepciones de forma individual en cada controlador. De este modo, se consigue un tratamiento uniforme de los errores y se mejora tanto la mantenibilidad del código como la claridad de las respuestas enviadas al cliente.

En particular, se definen dos métodos de manejo de excepciones. El primero captura instancias de `MethodArgumentNotValidException`, que se generan cuando fallan las validaciones de los datos de entrada en las peticiones HTTP. En este caso, se construye un mapa que asocia cada campo inválido con su correspondiente mensaje de error, devolviendo una respuesta estructurada junto con un código HTTP 400 (Bad Request). El segundo método gestiona excepciones genéricas de tipo `RuntimeException`, devolviendo igualmente un código 400 y un mensaje de error simplificado. Esta aproximación permite ofrecer respuestas más

comprensibles y consistentes al usuario, además de facilitar la depuración y el control de errores en la aplicación.

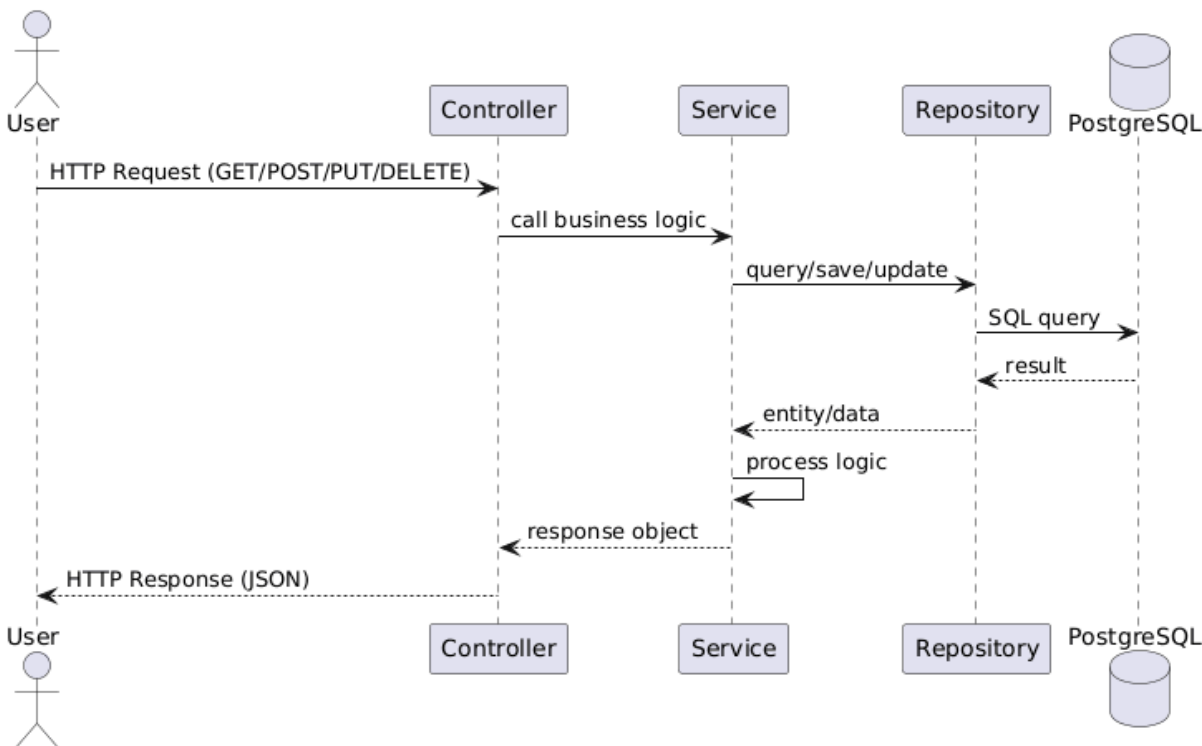
Actualmente, el sistema de gestión de errores presenta un nivel básico de implementación, por lo que se plantea como mejora futura un mayor grado de detalle en la clasificación de excepciones, así como la adaptación de las respuestas a las buenas prácticas en el uso de códigos de estado HTTP. Asimismo, se prevé la incorporación de nuevos tipos de errores conforme se implementen funcionalidades adicionales, especialmente en lo relativo a los mecanismos de autenticación y autorización.

4. Diagramas de secuencia

Los diagramas de secuencia constituyen una herramienta fundamental para la representación del comportamiento dinámico del sistema, permitiendo visualizar de manera clara la interacción entre los distintos componentes de la aplicación a lo largo del tiempo. En el contexto de esta API, estos diagramas resultan especialmente útiles para describir el flujo de ejecución de las peticiones realizadas por los usuarios y cómo estas son procesadas a través de las diferentes capas del sistema.

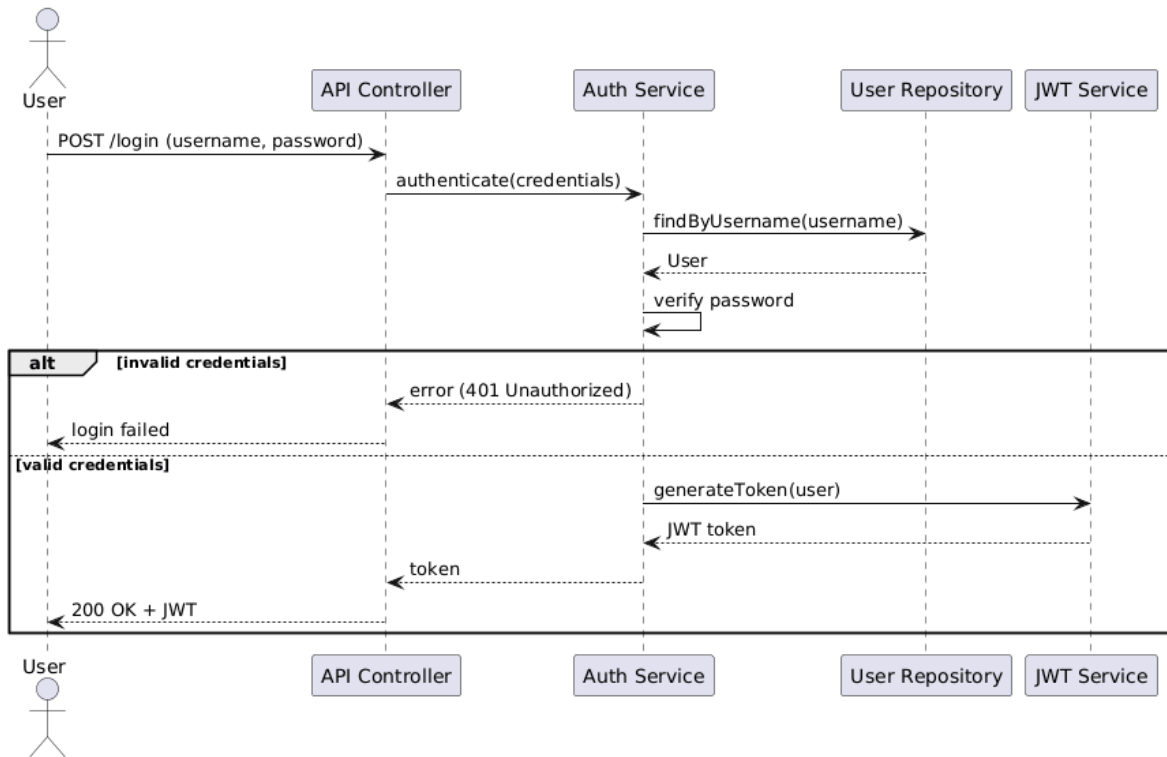
En concreto, los diagramas de secuencia elaborados reflejan la comunicación entre los principales elementos de la arquitectura basada en Spring Framework, como los controladores, servicios, repositorios y la base de datos. A través de ellos, se puede observar cómo una petición HTTP es recibida por el controlador, delegada al servicio correspondiente para la aplicación de la lógica de negocio, y posteriormente gestionada por el repositorio para el acceso a los datos, finalizando con la devolución de una respuesta al cliente.

4.1 Petición HTTP

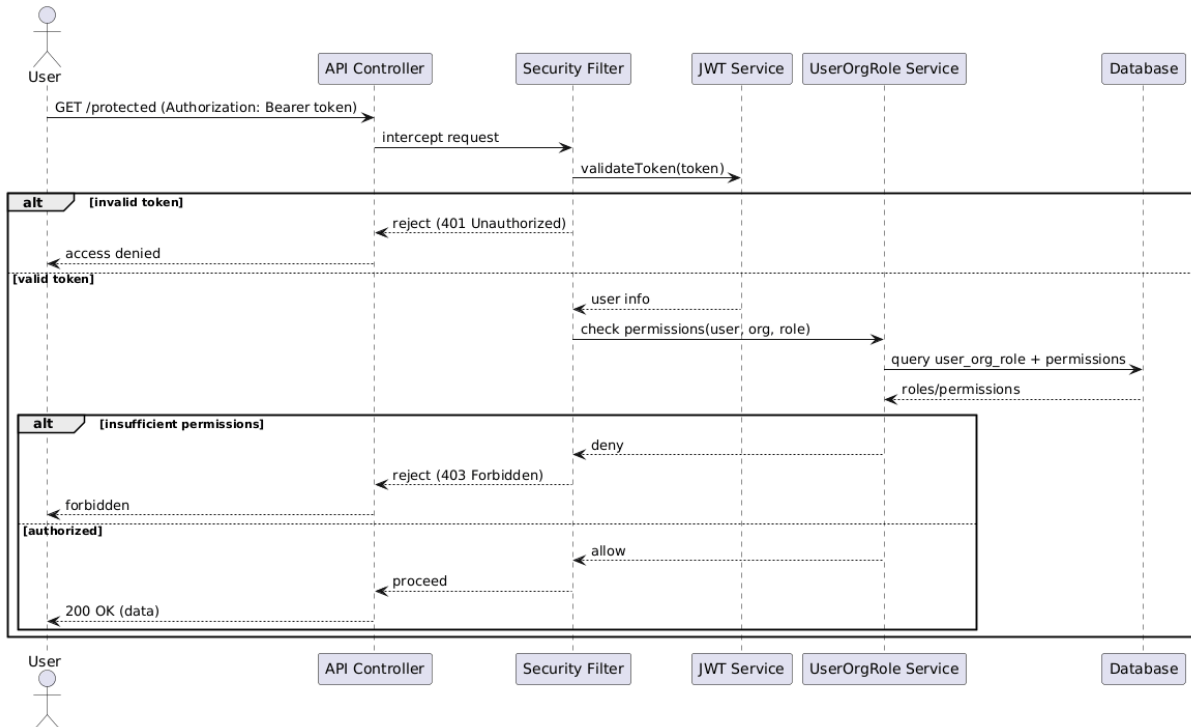


Asimismo, se ha considerado oportuno elaborar diagramas adicionales de los distintos elementos a implementar, dado que en esta fase del proyecto se dispone de un conocimiento más consolidado y preciso sobre el funcionamiento y las prácticas de desarrollo en Spring Framework. Estos diagramas se detallan a continuación

4.2 Autenticación de usuario



4.3. Autorización de acciones



5. Conclusiones provisionales

En esta fase, el proyecto ha evolucionado desde una base teórica hacia la implementación funcional del sistema, consolidando los principales componentes de la arquitectura.

Entre los avances más relevantes destaca la consolidación de la arquitectura en capas, así como la implementación de las principales entidades que conforman el sistema. Asimismo, se ha completado la integración con la base de datos, garantizando la persistencia y correcta gestión de la información. De forma complementaria, se ha mejorado la robustez de la aplicación mediante la incorporación de validaciones y una gestión más estructurada de los errores.

En cuanto al trabajo futuro, se contempla la implementación de un sistema de autenticación basado en JWT, así como la integración completa de la seguridad mediante Spring Security. Además, se desarrollará el sistema de autorización, se completarán los endpoints pendientes y se llevará a cabo un proceso de pruebas y pulido global del sistema.

6. Fuentes de información consultadas

[1] Walls, C. (2019). *Spring in Action* (5th ed.). Manning Publications.

[2] Spring Guides. (2024). *Building web applications with Spring*. <https://spring.io/guides>

[3] OpenAPI Initiative. (2021). *OpenAPI Specification v3.1.0*. <https://spec.openapis.org/oas/v3.1.0>

[4] Eclipse Foundation. (2022). *Jakarta Persistence Project: Specification v3.1*. Jakarta EE. <https://jakarta.ee/specifications/persistence/3.1/>

[5] Eclipse Foundation. (2024). *Jakarta Bean Validation Specification v3.1*. Jakarta EE. <https://jakarta.ee/specifications/bean-validation/3.1/>

[6] Pivotal Software. (2020). *Spring MVC Framework: @ControllerAdvice & Exception Handling*. Spring.io. <https://docs.spring.io/spring-framework/docs/current/reference/html/web.html#mvc-ann-controlleradvice>

7. Uso de la IA generativa en el desarrollo del informe y del trabajo

Durante el desarrollo del proyecto se ha hecho uso de herramientas de IA generativa como apoyo en distintas tareas:

- Resolución de dudas técnicas relacionadas con Spring Boot y arquitectura backend.
- Asistencia en la redacción y estructuración de documentación.

El uso de estas herramientas ha sido complementario, empleándose como apoyo al aprendizaje y desarrollo, sin sustituir la comprensión ni la implementación propia de las funcionalidades del sistema.

Se ha prestado especial atención a la validación y adaptación del contenido generado, asegurando su corrección y adecuación al contexto del proyecto.